

1993 Tennessee Eastman Süreci İçin Gymnasium Tabanlı Pekiştirmeli Öğrenme Ortamı: Model, Çözücü ve Uygulama Araştırması

Yönetici özeti

Ana sonuç: 1993 Downs–Vogel Tennessee Eastman (TE) modelini sadık biçimde Gymnasium'a taşımak için bir DAE çözücüsü zorunlu değildir. Orijinal dağıtımın dış arayüzü, `TEFUNC` tarafından 50 durumun türevlerini döndüren **sürekli-zamanlı bir ODE başlangıç-değer problemidir**; bu 50 durum yaklaşık olarak dört adet sekiz-bileşenli holdup grubu, dört enerji durumu, iki soğutma-suyu durumu ve on iki valf/aktüatör dinamiğinden oluşur. Termodinamik denge, yoğunluk, entalpi ve akış ilişkileri RHS içinde cebirsel olarak değerlendirilir; integratöre $F(t, y, \dot{y})=0$ biçiminde bir DAE verilmez. Bu nedenle orijinal model için “DAE indeksi” **uygulanamaz**. Downs ve Vogel ayrıca modeli özellikle aşırı sertliği azaltacak şekilde kurmuş, nominal noktadaki en hızlı özdeğeri yaklaşık $-1968 h^{-1}$, yani yaklaşık 1.8 s zaman sabiti olarak bildirmiş ve örnek programlarında **1 s sabit-adımlı Euler** kullanmıştır. ¹

Bununla birlikte, “**Euler 1 s çalışıyor**” ile “**Euler en doğru çözücüdür**” aynı şey değildir. Nominal lineer mod için $h = 1$ s olduğunda $h\lambda \approx -0.547$; bu, açık Euler'in negatif reel eksenindeki kararlılık bölgesinin içindedir. Yalnızca bu mod dikkate alındığında teorik kararlılık sınırı yaklaşık 3.66 s'dir; 2015 Bathelt–Ricker–Jelali çalışması da pratikte Euler için 1 s'nin yaklaşık üst sınır olduğunu ve daha büyük adımların kararsızlaşabildiğini gözlemlemiştir. Bu, 1 s Euler'in tarihsel seçimini iyi açıklar; fakat birinci mertebe global hata, arıza bölgeleri ve güvenlik sınırları nedeniyle onu araştırma amaçlı “ground truth” çözümü yapmaz. ²

Bu nedenle önerim iki katmanlıdır:

Kullanım	Önerilen varsayılan	Neden
Fiziksel doğruluk / makale sonuçları / değerlendirme	SUNDIALS CVODE-BDF ; bağımsız referans olarak SciPy <code>Radau</code>	Stiff ve non-stiff bölgelere karşı sağlam; değişken adım/mertebe; kök bulma; Jacobian ve sparse/Krylov seçenekleri; TE'nin ODE yapısıyla doğrudan uyumlu. ³
Saf Python prototip	SciPy <code>BDF</code> veya <code>Radau</code>	Kurulum kolay, iyi hata kontrolü, Jacobian/sparsity ve event desteği. ⁴
Yüksek hacimli RL eğitimi	Sabit-adımlı RK4, 1 s iç adım ; hız için doğrulandıktan sonra daha büyük adım	Deterministik maliyet, kolay vektörleştirme; Euler'e göre çok daha yüksek mertebe; adaptif çözücünün RNG ile etkileşimini önler. Bu raporun mühendislik önerisidir.

Kullanım	Önerilen varsayılan	Neden
Legacy sonuçlarla birebir uyum	Euler, 1 s	Downs-Vogel örnek akışı ve birçok sonraki Fortran türevinin doğal referansı. ⁵
Gerçek DAE/Modelica yeniden formülasyonu	IDA/IDAS	SUNDIALS IDA, $F(t, y, \dot{y}) = 0$ DAE'ler için değişken mertebeli BDF 1-5 kullanır ve özellikle yarı-açık index-1 sistemlerde tutarlı başlangıç koşulu hesaplayabilir. ⁶
Diferansiyellenebilir simülasyon/NMPC	CasADi + CVODES/IDAS; JAX tarafında Diffrax	CasADi otomatik türev/Jacobian ve SUNDIALS arayüzlerini sağlar; Diffrax JAX içinde implicit stiff yöntemler sunar. ⁷

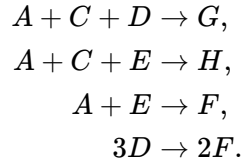
En kritik uygulama kararı çözücünden önce **rastlantısallığı RHS'den ayırmaktır**. Orijinal TE kodunun random process disturbance ve measurement-noise üretimi `tefunc()` çağrılarıyla bağlantılıydı. Adaptif RK/BDF çözücüleri aynı fiziksel zaman aralığında farklı sayıda RHS değerlendirmesi yaptığı için aynı seed farklı disturbance yolları üretebiliyordu. 2015 revizyonunun temel nedeni budur: disturbance RNG'sini çözücü çağrı sayısından ayırarak Euler, ode3 ve ode45 sonuçlarını tekrar solver-bağımsız hâle getirmiştir. RL ortamında da **plant RHS saf/deterministik**, disturbance/noise scheduler ise ayrı ve fiziksel zamanla tetiklenen bir alt sistem olmalıdır. ⁸

Kamuya açık uygulamalar açısından önemli bir ayrım vardır. `AIRI-Institute/iprl_benchmark` gerçekten Gym/Gymnasium uyumlu bir **Simplified Tennessee Eastman Process (STEP)** sunmaktadır, fakat bu tam 1993 50-state TE modeli değildir. `pyTEP` yüksek seviyeli Python API sağlar ama MATLAB/Simulink motoruna dayanır. `tep2py` ve `jkitchin/tennessee-eastman-profbraatz` ise Fortran/Python üzerinden orijinal modele çok daha yakındır fakat hazır Gymnasium ortamları değildir. COSTEP ise modern, açık bir Simulink TE modelidir ve dokümantasyonunda `ode45` ile stiff sistemler için `ode15s` gibi seçenekler gösterilir. İncelenen önde gelen kamu projeleri arasında **“tam 1993 modeli + modern Gymnasium API + açıkça modüler ve doğrulanmış stiff solver backend” kombinasyonunu hazır sağlayan bir repo bulunmamaktadır**; bu nedenle yeni bir wrapper/ortam katmanı oluşturmak en temiz yaklaşımdır. ⁹

Kullanıcının istediği RL arayüzüne ilişkin **kesin action space, observation space, reward fonksiyonu, action-hold/control periyodu ve episode horizon belirtilmemiştir**. Orijinal proses 12 manipulated variable ve 41 measurement sağlar; bunların tamamının RL aksiyonu/gözlemi yapılması zorunlu değildir. Özellikle gerçek ölçümleri kullanırsanız latent 50-state durum tamamen gözlenmediğinden problem POMDP niteliği kazanabilir. ¹⁰

Modelin fiziksel ve matematiksel temeli

Downs ve Vogel'in 1993 problemi, reaktör, ürün kondenseri, buhar-sıvı separatörü, recycle kompresörü ve stripper içeren bir sürekli kimyasal tesistir. Sekiz bileşen A-H kullanılır. Dört temel reaksiyon şunlardır:



Tümü geri dönüşümsüz ve ekzotermiktir. Makale hızların Arrhenius sıcaklık bağımlılığı taşıdığını, ana reaksiyonların yaklaşık birinci mertebeli davranış gösterdiğini ve G üretim reaksiyonunun daha yüksek aktivasyon enerjisine sahip olduğunu belirtir. Reaktör iyi karıştırılmış, katalizör uçucu olmayan ve sıvı fazda bulunan bir sistem olarak modellenmiştir. ¹¹

Orijinal makale, modern anlamda tek bir uzun sembolik denklem seti vermek yerine Fortran `TEINIT` / `TEFUNC` kodunu **modelin yürütülebilir spesifikasyonu** olarak dağıtmıştır. `TEFUNC` 50 durumun türevlerini ve proses ölçümlerini hesaplar; kullanıcı integrasyon algoritmasını sağlamalıdır. Örnek `TEMAIN` ise kontrol hesabı, ölçüm/valf kayıtları ve integrasyonu ayrı adımlar halinde yürütür ve örnekte Euler kullanır. ¹²

Kaynak koddaki reaksiyon kinetikleri makalenin sözel “yaklaşık birinci derece” tanımından daha ayrıntılıdır. Uygulama düzeyinde, sıcaklık Kelvin cinsinden alınarak aşağıdaki yapıya sahiptir:

$$\begin{aligned}
r_1 &= \exp\left(31.5859536 - \frac{40000}{1.987T}\right) p_A^{1.1544} p_C^{0.3735} p_D V_{\text{eff}}, \\
r_2 &= \exp\left(3.00094014 - \frac{20000}{1.987T}\right) p_A^{1.1544} p_C^{0.3735} p_E V_{\text{eff}}, \\
r_3 &= \exp\left(53.4060443 - \frac{60000}{1.987T}\right) p_A p_E V_{\text{eff}}, \\
r_4 &= 0.767488334 \exp\left(53.4060443 - \frac{60000}{1.987T}\right) p_A p_D V_{\text{eff}}.
\end{aligned}$$

Buradaki önemli nüans, dördüncü reaksiyonun stokiyometrik olarak $3D \rightarrow 2F$ olmasına rağmen uygulama hız ifadesinin $p_A p_D$ 'ye bağlı olmasıdır; A bu reaksiyonun stokiyometrik tüketim terimine girmez. Kodun reaksiyon kaynak terimleri kabaca

$$\begin{aligned}
R_A &= -r_1 - r_2 - r_3, & R_B &= 0, \\
R_C &= -r_1 - r_2, & R_D &= -r_1 - \frac{3}{2}r_4, \\
R_E &= -r_2 - r_3, & R_F &= r_3 + r_4, \\
R_G &= r_1, & R_H &= r_2
\end{aligned}$$

şeklinde. Tam bir modern yeniden yazımda, yalnızca 1993 makalesindeki şematik reaksiyonlardan hareket etmek yerine bu kod ifadelerinin regression testlerle korunması gerekir. ¹³

Dört proses holdup bloğundaki bileşen denkliklerinin ortak biçimi

$$\frac{dn_{i,j}}{dt} = \sum_{\text{in}} F_{i,\text{in}} - \sum_{\text{out}} F_{i,\text{out}} + \sum_r \nu_{ir} r_r$$

ve enerji denkliklerinin ortak biçimi

$$\frac{dU_j}{dt} = \sum_{\text{in}} F_{\text{in}} H_{\text{in}} - \sum_{\text{out}} F_{\text{out}} H_{\text{out}} + Q_j + \sum_r (-\Delta H_r) r_r$$

olarak görülebilir. Kaynak kodda sekiz bileşen için dört sekizli blok, bunlarla ilişkili dört enerji durumu, iki cooling-water sıcaklık durumu ve on iki valf durumu türetilir. Valf dinamikleri birinci merteye biçimdedir:

$$\dot{z}_m = \frac{v_m^{cmd} - z_m}{\tau_m}, \quad m = 1, \dots, 12.$$

Böylece aktüatör dinamiği plant state'in içindedir; RL aksiyonu doğrudan fiziksel akış yerine valf/manipulated-variable komutu olarak modellenenebilir. ¹⁴

Termodinamik taraf da modelin sayısal davranışı açısından önemlidir. Kondanse olabilen bileşenlerin buhar basıncı kaynak kodda doğal üstel biçimde Antoine-tipinde,

$$P_i^{sat}(T) = \exp \left(A_i + \frac{B_i}{T + C_i} \right),$$

sıvı yoğunluğu

$$\rho_i(T) = A_{\rho,i} + B_{\rho,i}T + C_{\rho,i}T^2,$$

ve sıvı entalpisi

$$H_{L,i}(T) = A_{H,i}T + \frac{B_{H,i}}{2}T^2 + \frac{C_{H,i}}{3}T^3$$

gibi polinomlarla hesaplanır; gaz ısı kapasiteleri de sıcaklık polinomlarıyla temsil edilir. Orijinal model ideal-gaz varsayımı, Raoult kanunu ve Antoine buhar basınçları kullanır; bütün kaplar well-mixed kabul edilir ve spatial/distributed parameter modeli yoktur. ¹⁵

Bu varsayımlar Gymnasium implementasyonu için doğrudan sonuç doğurur:

Orijinal varsayım	Sayısal/ortam etkisi
Lumped, well-mixed vessels	PDE çözücü gerekmez; 50-state ODE yeterlidir. ¹⁶
Ideal gas + Raoult + Antoine	Termodinamik hesaplar RHS içindeki cebirsel constitutive relations olarak tutulabilir. ¹⁷
Exothermic Arrhenius reactions	Sıcaklığa kuvvetli nonlinear hassasiyet, özellikle fault bölgelerinde lokal stiffness yaratabilir. ¹⁸
Pressure-dependent valve/flow ilişkileri ve compressor curve	Basınç-akış coupling'i güçlü nonlinear davranış oluşturur; safety-event kontrolü gerekir. ¹⁹
Gaussian measurement noise ve stochastic disturbances	RNG, adaptif RHS çağrılarından kesinlikle ayrılmalıdır. ²⁰
Model startup/shutdown simülasyonu için tasarlanmamıştır; shutdown limits vardır	RL episode termination fiziksel operating-envelope ile sınırlandırılmalıdır. ¹⁹
Çok hızlı transmitter lag'leri ve bazı hızlı dinamikler bilinçli olarak sadeleştirilmiştir	Model gerçek tesisin bütün yüksek frekanslı dinamiklerini temsil etmez; daha ayrıntılı model daha stiff olabilir. ¹⁹

Modern varyantların üçü özellikle önemlidir. Bathelt–Ricker–Jelali 2015 revizyonu fiziksel/kimyasal çekirdeği değiştirmekten çok random disturbance üretimini solver-call-count'tan ayırmış, `tefunc()`

yürütmesini output ve derivative hesapları açısından yeniden yapılandırmış, ek ölçümler eklemiş ve raporlanan testlerde çalışma süresini yaklaşık %45 azaltmıştır. ²¹ Modelica çalışması iki yeniden kullanılabilir model, `TEprocess` ve `TESimplified`, sunmuş ve bunları literatür verileriyle doğrulamıştır. ²² Daha yeni COSTEP projesi ise TE'yi açık Simulink blokları halinde yeniden kurarak yapı ve parametrelerin daha görünür olmasını amaçlar. ²³

Buna karşılık Jockenhövel, Biegler ve Wächter'in dinamik optimizasyon çalışmasında TE yaklaşık **200 differential-algebraic equation** içeren farklı bir first-principles/NMPC formülasyonuna dönüştürülmüştür. Bu, "TE her zaman 200-equation DAE'dir" anlamına gelmez; 1993 simulator interface'i ile optimizasyon için oluşturulan DAE formülasyonu farklı modellerdir. Erişilebilir birincil açıklamada bu DAE'nin diferansiyel indeksi açıkça belirtilmediğinden index-1 olduğu varsayılmamalıdır. ²⁴

Zaman formülasyonu, sertlik ve DAE meselesi

Fiziksel plant sürekli zamandadır:

$$\dot{x} = f(x, u, d, t), \quad y = h(x, u, d, t) + \eta.$$

Gymnasium ise doğal olarak sampled-data geçişi oluşturur. Sabit tutulan bir RL aksiyonunda

$$x_{k+1} = \Phi_{\Delta t_c}(x_k, u_k, d),$$

burada Φ , sürekli ODE'nin Δt_c control interval boyunca sayısal akışıdır. Dolayısıyla **Gymnasium ortamı için plant'i baştan discrete equation olarak türetmek gerekmez**: `step(action)` içerisinde aksiyon zero-order hold ile tutulur ve continuous plant daha küçük iç adımlarla entegre edilir. `reset()` seed ve başlangıç durumunu, `step()` ise gözlem/reward/termination güncellemelerini yönetir; Gymnasium ayrıca özel ortamların `check_env` ile doğrulanmasını önerir. ²⁵

Bu ayrım önemlidir çünkü örneğin 30 s'lik bir RL action interval seçilmiş olsa bile fizik modeli 30 s Euler ile ilerletmek gerekli değildir. Doğru yapı

$$\Delta t_c = 30 \text{ s}, \quad h_{\text{internal}} = 1 \text{ s}$$

olup her RL adımında 30 integrator substep çalıştırmaktır. **Ancak bu örnekteki 30 s yalnızca açıklama amaçlıdır**: kullanıcının control/action sample interval'i belirtilmemiştir. Aynı şekilde hangi 12 manipulated variable'in ajan tarafından kullanılacağı, 41 ölçümün hangilerinin observation olacağı ve reward tanımı da belirtilmemiştir. Orijinal prosesin 12 MV ve 41 measurement sağlaması bu RL seçimlerini otomatik olarak belirlemez. ¹⁰

Orijinal model ODE'dir. `TEINIT` başlangıç durumlarını sağlar; `TEFUNC` doğrudan $\dot{x}$ döndürür; integrator bunu ilerletir. Raoult/VLE, akış, basınç, sıcaklık ve entalpi hesaplarının RHS içinde cebirsel olması modelin integratör açısından DAE olduğu anlamına gelmez. DAE olması için çözücüye doğrudan

$$F(t, y, \dot{y}) = 0$$

residual'ı verilmesi ve bir kısım değişkenlerin cebirsel constraints ile birlikte çözülmesi gerekir. SUNDIALS IDA tam olarak bu ikinci problem sınıfını hedefler. ²⁶

Buna göre sınıflandırma şöyledir:

Formülasyon	Matematiksel sınıf	DAE indeksi	Uygun çözücü
Downs-Vogel TEFUNC, 1993	Explicit RHS ODE, 50 state	Uygulanamaz	CVODE, SciPy BDF/Radau/RK, Euler/RK4
Ricker/Bathelt kod ailesi	Esasen aynı continuous ODE + yeniden düzenlenmiş stochastic/execution katmanı	Uygulanamaz	Aynı ODE çözücüleri ²⁷
Modelica TE formülasyonları	Tool tarafından ODE/DAE'ye structural processing uygulanabilir	Model ve compiler'a bağlı	DASSL/IDA vb.; OpenModelica'nın genel varsayılanlarından biri DASSL'dir. ²⁸
Jockenhövel-Biegler-Wächter optimizasyon formülasyonu	Yaklaşık 200 DAE	Erişilebilir kaynakta belirtilmemiş	DAE integrator veya direct transcription/NLP. ²⁴

Stiffness konusunda en doğru tanım “hafif/orta derecede ve çalışma noktasına bağlı stiffness”tir.

Downs-Vogel modelinin geliştirilmesinde gas-pressure dynamics kaynaklı aşırı stiffness bilinçli olarak sınırlandırılmıştır; nominal noktadaki en hızlı eigenvalue $-1968 h^{-1}$ olarak verilmiş ve 1 s integrasyon adımının yeterli olduğu belirtilmiştir. Aynı tesiste inventory/composition/thermal davranışının çok daha uzun zaman ölçeklerinde gerçekleşmesi nedeniyle yine de çok-zaman-ölçekli bir sistem vardır. ¹⁹

Euler için lineer test denklemi $x' = \lambda x$ üzerinden:

$$h|\lambda| = \frac{1}{3600} \times 1968 \approx 0.547,$$

ve negatif reel eigenvalue için Euler stability condition yaklaşık

$$-2 < h\lambda < 0$$

olduğundan nominal hızlı mod 1 s'de karardır. Yalnız bu özdeğer dikkate alındığında

$$h_{\max} \approx \frac{2}{1968} h \approx 3.66 s.$$

Ancak nonlinear reaction rates, valve saturation, compressor ilişkileri ve disturbance/fault bölgeleri nedeniyle bu 3.66 s değeri plant genelinde güvenli bir integrator step limiti değildir. Nitekim 2015 karşılaştırması 1 s'nin Euler açısından yaklaşık pratik üst sınır olduğunu bildirir. ²⁹

2015 çalışmasının belki de solver seçiminden daha önemli sonucu şudur: orijinal stochastic modelle Euler, Bogacki-Shampine ve Dormand-Prince aynı physical disturbance altında görünürde çok farklı trajectories üretmiştir; disturbance'lar kapatıldığında solver dependency kaybolmuştur. Sebep fizik denklemleri değil, RNG state'inin `tefunc()` çağrı sayısına bağlanmasıdır. Adaptif bir solver'ın inner integration loop'u daha çok RHS çağrısı yaptığı için random-number stream de değişmiştir. Revize model disturbance RNG'si ile measurement-noise RNG'sini ayırır ve disturbance polynomials'ın güncellemesini fiziksel zamana bağlar. ³⁰

Bu nedenle Gymnasium tasarımında ideal ayrışım şöyledir:

flowchart LR

```
A[RL ajanı] -->|action u_k| B[Gymnasium Env]
B --> C[Action adapter + ZOH]
C --> D[Deterministik TE RHS f(x,u,d)]
E[Disturbance scheduler<br/>fiziksel zaman tabanlı RNG] --> D
D --> F[ODE/DAE solver]
F --> G[Latent state x]
G --> H[Measurement model]
I[Measurement-noise RNG<br/>yalnız observation tick'lerinde] --> H
H --> J[Observation]
G --> K[Safety / shutdown events]
K --> B
J --> B
B -->|obs, reward, terminated,<br/>truncated, info| A
```

Bu mimaride `rhs(t, x)` **asla random sayı çekmez**. Böylece Euler, RK4, Radau veya CVODE aynı disturbance realization altında karşılaştırılabilir; solver değiştiğinde fiziksel input path değişmez. Bu tasarım 2015 revizyonunun temel tekrarlanabilirlik ilkesini doğrudan RL ortamına taşır. ³¹

Sayısal çözücüler ve parametre seçimi

SciPy'nin güncel `solve_ivp` arayüzü explicit `RK23`, `RK45`, `DOP853`; implicit `Radau` ve `BDF`; ayrıca stiffness'i otomatik algılayıp Adams/BDF arasında geçen `LSODA` yöntemlerini sağlar. SciPy explicit RK yöntemlerini non-stiff, Radau/BDF'yi stiff problemler için önerir; BDF merteye 1-5, Radau IIA merteye 5'tir. ⁴ SUNDIALS CVODE ise non-stiff problemler için değişken mertebeli Adams-Moulton 1-12 ve stiff problemler için BDF 1-5 sağlar. ³²

Çözücü karşılaştırması

Çözücü	Tip	TE-1993 için değerlendirme	Maliyet / avantaj / risk
Euler, fixed 1 s	Explicit, 1. merteye	Tarihsel referans ve hızlı RL baseline	1 RHS/step; en ucuz. Fakat accuracy zayıf, daha büyük step'e toleransı düşük. Downs-Vogel örneği ve 2015 testi 1 s kullanımını destekler. ³³
RK4, fixed 0.5-1 s	Explicit, 4. merteye	Önerilen yüksek-throughput eğitim seçeneği	4 RHS/step; deterministic execution; Euler'e göre çok daha düşük discretization error. Bu raporun önerisidir.
RK45	Adaptive explicit RK 5(4)	Nominal bölgede kullanılabilir	Kolay; ancak stiffness olduğunda çok sayıda step alabilir. ³⁴

Çözücü	Tip	TE-1993 için değerlendirme	Maliyet / avantaj / risk
DOP853	Adaptive explicit RK8	Smooth ve kanıtlanmış non-stiff bölgelerde yüksek accuracy	SciPy yüksek hassasiyet için DOP853'ü explicit yöntemler arasında önerir. ³⁴
Radau	Implicit Radau IIA order 5	Bağımsız high-accuracy oracle olarak çok iyi	Stiff stability güçlü; her step'te nonlinear/linear solve nedeniyle Euler/RK'den pahalı. ³⁴
SciPy BDF	Implicit multistep 1-5	Saf-Python araştırma default'u	Stiff bölgede verimli; Jacobian vermek önemli. ³⁵
LSODA	Adams ↔ BDF otomatik	Kullanışlı fallback	Stiffness detection otomatik; ancak eski ODEPACK Fortran wrapper ve sparse Jacobian API'si BDF/Radau kadar esnek değildir. ³⁶
CVODE-BDF	Variable-step/ order implicit BDF 1-5	Önerilen üretim / research solver	Newton + dense/sparse/Krylov seçenekleri, rootfinding, restart, tolerans kontrolü. ³
CVODE-Adams	Variable-step/ order implicit Adams-Moulton 1-12	Non-stiff olduğu kanıtlanan senaryolarda	CVODE'nun resmi non-stiff seçeneği. ³⁷
IDA/IDAS	DAE BDF 1-5	Orijinal TE için gereksiz; gerçek DAE reformülasyonunda doğru seçim	Consistent initial conditions ve DAE residual çözümü sağlar. ⁶
CasADi-CVODES/IDAS	AD + SUNDIALS	Differentiable sensitivity/ NMPC için güçlü	Jacobian/sensitivity otomatik oluşturulabilir. ³⁸
DiffraX Kvaerno*	JAX implicit RK	JAX tabanlı differentiable stiff simulator	DiffraX stiff ODE'lerde implicit yöntemleri örnekler ve stiff/non-stiff solver ayrımını açıkça destekler. ³⁹

Research “truth” çözümünde önerdiğim başlangıç ayarı, state'leri ölçekledikten sonra CVODE-BDF veya SciPy Radau/BDF için $rtol=1e-7$ – $1e-8$, scaled $atol=1e-9$ – $1e-10$, başlangıçta $max_step=1$ s ve safety/root event'lerini açık tutmaktır. Bu sayılar Downs-Vogel tarafından verilmiş solver parametreleri değildir; TE için önerdiğim validation-oriented başlangıç noktalarıdır. SciPy local error'ı $atol + rtol*abs(y)$ kriteriyle kontrol eder ve farklı ölçekli state'ler için component-wise $atol$ kullanımını destekler. SUNDIALS da vector absolute tolerances sağlar ve genel olarak $reltol>10^{-3}$ kullanımını önermediğini belirtir. ⁴⁰

Raw TE state'lerinin farklı fiziksel birimleri olduğu için tek bir scalar $atol=1e-8$ kullanmak ideal değildir. Daha iyi yaklaşım

$$\tilde{x}_i = \frac{x_i - x_i^{ref}}{s_i}$$

gibi nondimensionalization yapıp toleransları $\tilde{x}$ uzayında uygulamak ya da

$$atol_i = \epsilon_{abs} s_i$$

şeklinde state-wise absolute tolerance vermektir. Bu, mole inventory, energy ve actuator states gibi farklı büyüklükte değişkenlerin hata testine adil katkı yapmasını sağlar; SciPy ve CVODE her ikisi de vector absolute tolerances'ı destekler. ⁴⁰

Jacobian seçimi: SciPy, Radau ve BDF için analitik/callable Jacobian sağlamayı finite-difference varsayıma tercih ettiğini açıkça belirtir; `jac_sparsity` da her satırda az sayıda nonzero varsa finite-difference Jacobian hesaplamasını önemli ölçüde hızlandırabilir. ³⁵ Bununla birlikte TE yalnızca 50 state olduğundan ilk implementasyonda dense 50×50 Newton matrix'i oldukça küçüktür; sparse altyapı overhead'i bazen kazancı aşabilir. Dolayısıyla benim sıralamam **dense exact/AD Jacobian → dense finite difference → sparsity yalnız profiling kazanç gösterirse** şeklindedir. Controller, estimator veya büyük augmentations eklenirse sparse/Krylov yaklaşımı daha anlamlı olur. CVODE matrix-based Newton interface'inde Jacobian ve $I - \gamma J$ matrislerini doğrudan yönetir ve çeşitli SUNLinearSolver seçeneklerine sahiptir. ⁴¹

Orijinal Fortran black-box RHS için analitik Jacobian yazmak emeklidir; 50 state için başlangıçta finite differences mantıklıdır. Saf Python/CasADi yeniden yazımında ise otomatik differentiation çok daha çekicidir. CasADi, CVODES/IDAS interface'inde BDF'nin ihtiyaç duyduğu Jacobian bilgisini expression graph'tan oluşturabilir ve forward/adjoint sensitivities sağlayabilir. ³⁸

Event handling özellikle shutdown açısından önemlidir. SciPy `solve_ivp` event fonksiyonlarının sıfır geçişlerini root finding ile belirleyebilir; ancak tek step içinde birden fazla crossing varsa bunlar kaçabilir; dolayısıyla `max_step` güvenlik eşiklerine göre sınırlandırılmalıdır. ³⁵ CVODE da rootfinding ve stop-time mekanizmaları sağlar. SUNDIALS ayrıca RHS'deki bilinen jump discontinuities için step'i discontinuity noktasında bitirip gerekirse `CVodeReInit()` ile yeniden başlatmayı önerir. Bu, Gym action'ın step boundary'de değişmesi açısından doğrudan önemlidir. ⁴²

Pratik ayarlarım şöyledir:

Profil	<code>rtol</code>	<code>atol</code>	Step kontrolü	Jacobian	Amaç
Oracle/ validation Radau	10^{-9}	scaled 10^{-11}	<code>max_step=0.5-1 s</code>	exact/FD	Convergence reference
Research CVODE-BDF	10^{-7}	scaled 10^{-9}	başlangıç <code>max_step=1 s</code> , sonra convergence ile gevşet	dense Newton	Yayın/ değerlendirme
Normal training BDF	10^{-5} – 10^{-6}	scaled 10^{-7} – 10^{-8}	1-5 s ancak benchmark sonrası	dense/FD	Accuracy-speed dengesi
RK4 training	—	—	fixed 1 s başlangıç	yok	Yüksek throughput

Profil	<code>rtol</code>	<code>atol</code>	Step kontrolü	Jacobian	Amaç
Legacy	—	—	Euler fixed 1 s	yok	1993-benzeri numerics
DAE/IDA	10^{-6} – 10^{-8}	scaled	adaptive BDF	residual Jacobian	Gerçek DAE variant

Bu değerler **önerilen başlangıç değerleridir; doğrulama sonucuna göre seçilmelidir**. Bathelt çalışmasında revised model için ode3'ün 4 s ve ode45'in bazı testlerde çok daha geniş adımlarla çalışabilmesi, accuracy requirements ve senaryo değişmeden “maksimum güvenli step” olarak yorumlanmamalıdır. Özellikle RL'de control discontinuities ve fault onset'leri ayrıca çözülmelidir. ⁴³

Kamu uygulamaları ve repository araştırması

24 Eylül 2026 itibarıyla incelenen belirgin TE kod aileleri, tam 1993 fiziksel model ile Gym/Gymnasium ergonomisinin henüz aynı projede olgun biçimde birleşmediğini gösteriyor.

Proje / kaynak	Model kapsamı	Solver yaklaşımı	Gym/ Gymnasium	Güçlü yanı	Ana sınırlama
Downs/Vogel → Braatz/Russell → <code>jkitchin/tennessee-eastman-profbraatz</code>	Tam 50-state TE, 41 measurement, 12 MV	Güncel Python wrapper'ın step döngüsü 1 s explicit Euler; Python ve Fortran RHS backend'leri	Hayır	Orijinal Fortran ailesine en yakın, test/data içeriği; Fortran backend repo tarafından yaklaşık 5–10× hızlı olarak raporlanıyor	Gym wrapper yok; adaptive solver'a geçirilecekse stochastic-state davranışı yeniden düzenlenmeli. ⁴⁴
<code>camaramm/tep2py</code>	Modified original Fortran TE	<code>f2py</code> ile <code>temain_mod.f</code> / <code>teprob.f</code> batch simulator	Hayır	Çok hafif Python–Fortran köprüsü	Modular solver API değil; README örnek binary'si eski Linux/Python nesline ait; RL step interface'i yok. ⁴⁵

Proje / kaynak	Model kapsamı	Solver yaklaşımı	Gym/ Gymnasium	Güçlü yanı	Ana sınırlama
ccreinartz11/pytep	TE simulator için yüksek-seviye scenario/API katmanı	İntegrasyon MATLAB/Simulink backend'e delege edilir	Doğrudan Gym değil	Interactive simulations, scenario configuration	Lisanslı MATLAB/ Simulink ve MATLAB Python engine gerektirir; dokümantasyon Python 3.7 ve MATLAB 2019a/ 2020b ile test edildiğini belirtir. ⁴⁶
AIRI-Institute/iprl_benchmark	Simplified Tennessee Eastman Process (STEP)	Node-based iplooper simulator; top-level README integrator algoritmasını açıkça belirtmiyor	Evet, Gym/ Gymnasium binding	RL deneyleri, online/offline RL, normalization/ wrapping altyapısı hazır	Tam 1993 TE değildir; fiziksel fidelity benchmark'ı olarak doğrudan kullanılamaz. ⁴⁷
COSTEP	Tam ve açık Simulink TE yeniden implementasyonu	Repo belgeleri ode45 ve stiff durumlarda ode15s örneklerini gösteriyor	Hayır	Blok seviyesinde şeffaf modern model, senaryo/data generation	MATLAB/ Simulink bağımlılığı; Gym için bridge gerekir. ²³
TEprocess / TESimplified Modelica	Ayrıntılı ve sadeleştirilmiş iki Modelica varyantı	Modelica toolchain ODE/ DAE solver seçer	Hayır	Reusable component modelleme, literatür verileriyle validation	DAE structure/ tool bağımlı; Python rollout overhead'i FMU/bridge'e bağlı. ²²
Bathelt-Ricker revised TE	Orijinal fizik + stochastic/ execution revizyonu	Euler/ode3/ ode45 karşılaştırılmış; solver-independent disturbances amaçlanmış	Hayır	Reproducibility açısından en önemli modern düzeltme	MATLAB/ Simulink/C-MEX kökenli workflow. ⁴⁸

[jkitchin/tennessee-eastman-profbraatz](#) şu anda en yararlı Python-native başlangıç noktalarından biridir. Repo tam 50 state, 41 measurement, 12 manipulated variable, 20 disturbance ve saf Python/opsiyonel Fortran backend sunar; README Fortran backend için yaklaşık 5–10× hız avantajı belirtir. ⁴⁹ Güncel simulator kodunda varsayılan internal increment 1/3600 h, yani 1 s'dir ve her `step()` sonunda

```
new_state = state + derivative * dt
```

mantığıyla explicit Euler yapılıdır. Dolayısıyla backend'in Fortran olması integrator'ın implicit hâle geldiği anlamına gelmez; Fortran/Python backend esas olarak RHS hesaplayıcısıdır. ⁵⁰

Bu mimari Gymnasium'a sarılması en kolay seçeneklerden biridir, fakat iki iyileştirme gerekir. Birincisi integrator interface'ini RHS'den ayırmak; ikincisi stochastic variables'ın solver substeps'ine bağımlı olmadığını regression test ile doğrulamaktır. Yalnızca mevcut Euler `step()` üzerinde Gym wrapper kurmak, hızlı ve tarihsel açıdan tutarlı bir baseline verir; fakat solver araştırması yapmak için yeterli değildir. 2015 çalışmasının random-number problemi burada özellikle dikkate alınmalıdır. ³¹

`tep2py` daha ince bir wrapper'dır. Repo, modified Fortran `temain_mod.f` ile `teprob.f` dosyalarını `numpy.f2py` üzerinden Python modülüne derler ve `tep.simulate()` ile batch simulation sağlar. README'deki interface dataset/data-generation iş akışına yöneliktir; integrator strategy kullanıcıya plugin olarak açılmamıştır. Bu nedenle Gym ortamı oluşturulabilir fakat action-at-each-step kontrolü ve solver swapping için `temain` seviyesinin altına inmek daha sağlıklı olur. ⁴⁵

`pyTEP` başka bir tasarım felsefesidir: Python'dan simulation scenario kurmayı kolaylaştırır fakat underlying MATLAB/Simulink engine'i gizler. Bu, araştırmacının solver'ı MATLAB tarafında değiştirebilmesini sağlasa da yüksek-throughput binlerce paralel RL environment açısından MATLAB-engine IPC ve lisanslama genellikle doğal bir sınırlamadır. PyTEP'in resmi dokümantasyonu lisanslı MATLAB/Simulink ile MATLAB Python engine gerektirdiğini doğrular. ⁵¹

AIRI projesi, incelenen kaynaklar içinde RL API'sine en doğrudan örnektir. `mgym` dizininde industrial-process simulators'ı Gym/Gymnasium-style env'e dönüştüren bindings bulunur; online ve offline RL için PPO, SAC, A2C/A3C gibi algoritma deneyleri ile çeşitli offline RL metodları yer alır. Ancak kullanılan proses açıkça **Simplified Tennessee Eastman Process** olarak adlandırılmıştır. Dolayısıyla bu repo Gym architecture, normalization ve RL experimentation tasarımı için örnektir; 1993 Downs-Vogel fizik doğruluğu için reference implementation değildir. Top-level dokümantasyonda STEP'in exact numerical integrator'ı belirtilmediğinden bu ayrıntı burada **belirtilmemiş** olarak kalmalıdır. ⁴⁷

COSTEP ise solver açısından ilginç bir modern kontrol noktasıdır. Repo dokümanı Simulink configuration örneklerinde `ode45` ve stiff sistemler için `ode15s` kullanımını önerir; model ayrıca random seed ayarını açıkça expose eder. Bu, "TE yalnız Euler ile çalıştırılabilir" düşüncesinin doğru olmadığını gösterir. Ancak COSTEP'in solver örneğini orijinal 1993 Fortran'ın resmi solver tercihi gibi yorumlamak doğru değildir; bu modern Simulink implementasyonunun konfigürasyonudur. ⁵²

Gymnasium mimarisi, Python araçları ve yüksek hızlı RL

En temiz Python tasarımı, TE modelini Gymnasium sınıfının içine gömmek yerine **plant-solver-stochastic scheduler-sensor-environment** katmanlarını bağımsız tutmaktır. Bu sayede aynı fiziksel model üzerinde Euler, RK4, SciPy BDF, CVODE ve daha sonra JAX/CasADi backend'leri aynı test suite'ini kullanabilir.

Önerilen paket yapısı:

```
te_gymnasium/  
├─ envs/
```

```

|   └─ te_env.py
├─ model/
|   └─ rhs.py           # deterministic 50-state TE equations
|   └─ thermo.py
|   └─ kinetics.py
|   └─ initial_conditions.py
├─ disturbances/
|   └─ scheduler.py     # process disturbances, physical-time clock
|   └─ measurement_noise.py # observation-time RNG
├─ solvers/
|   └─ base.py
|   └─ fixed_rk.py     # Euler / RK4
|   └─ scipy_ivp.py    # BDF / Radau / LSODA
|   └─ sundials.py     # CVODE / optional IDA
|   └─ casadi_backend.py
├─ control/
|   └─ action_adapter.py
├─ validation/
|   └─ reference_cases.py
└─ tests/
    └─ test_fortran_parity.py
    └─ test_solver_convergence.py
    └─ test_rng_invariance.py
    └─ test_gymnasium_api.py

```

Gymnasium'ın kendi araçları `reset(seed=...)` ile RNG yönetimini ve `check_env` ile action/observation spaces dahil temel API denetimlerini destekler. Çoklu rollout için `SyncVectorEnv` ve diğer vector-environment altyapıları mevcuttur. ⁵³

Minimal solver-independent ortam çekirdeği şu biçimde olmalıdır:

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Callable

import gymnasium as gym
import numpy as np
from scipy.integrate import solve_ivp

@dataclass(frozen=True)
class SolverConfig:
    method: str = "BDF"
    rtol: float = 1e-7
    atol: float | np.ndarray = 1e-9
    max_step_s: float = 1.0

class TennesseeEastmanEnv(gym.Env):

```

```

"""
Action/observation definitions are deliberately injected:
the user's exact RL action and observation spaces are unspecified.
"""

def __init__(
    self,
    rhs: Callable,
    initial_state: Callable,
    measurement_model: Callable,
    disturbance_scheduler,
    action_space: gym.Space,
    observation_space: gym.Space,
    control_dt_s: float,
    horizon_s: float,
    solver: SolverConfig = SolverConfig(),
):
    super().__init__()
    self.rhs = rhs
    self.initial_state = initial_state
    self.measurement_model = measurement_model
    self.disturbances = disturbance_scheduler

    self.action_space = action_space
    self.observation_space = observation_space

    self.control_dt_h = control_dt_s / 3600.0
    self.horizon_h = horizon_s / 3600.0
    self.solver = solver

    self.t = 0.0
    self.x = None

def reset(self, *, seed=None, options=None):
    super().reset(seed=seed)

    # Give stochastic subsystems independent streams derived from Env
seed.
    disturbance_seed = int(self.np_random.integers(0, 2**32 - 1))
    self.disturbances.reset(seed=disturbance_seed)

    self.t = 0.0
    self.x = np.asarray(self.initial_state(options), dtype=np.float64)

    obs = self.measurement_model(
        self.t, self.x, rng=self.np_random
    )
    return obs, {}

def step(self, action):
    u = self._map_action(action)

```

```

t_end = self.t + self.control_dt_h

# Important: no random-number generation inside plant_rhs.
d = self.disturbances.value_at(self.t)

def plant_rhs(t, x):
    return self.rhs(t, x, u, d)

result = solve_ivp(
    plant_rhs,
    (self.t, t_end),
    self.x,
    method=self.solver.method,
    rtol=self.solver.rtol,
    atol=self.solver.atol,
    max_step=self.solver.max_step_s / 3600.0,
)

if not result.success:
    raise RuntimeError(result.message)

self.x = result.y[:, -1]
self.t = t_end

obs = self.measurement_model(
    self.t, self.x, rng=self.np_random
)

terminated = self._physical_shutdown(self.x)
truncated = self.t >= self.horizon_h
reward = self._reward(self.x, u)

info = {
    "nfev": result.nfev,
    "njev": result.njev,
    "nlu": result.nlu,
    "physical_time_h": self.t,
}
return obs, reward, terminated, truncated, info

```

SciPy bu kullanımda BDF/Radau, tolerans, Jacobian, sparsity, event ve solver diagnostic counters (`nfev`, `njev`, `nlu`) sağlar. ⁵⁴ Ancak kod örneğindeki basitlik uğruna disturbance'ın control interval boyunca sabit tutulduğu görülmektedir; gerçek TE random-variation disturbance scheduler'ı interval içinde kendi update times'ına sahipse interval event noktalarında bölünmeli ve ayrı segmentler entegre edilmelidir. Bu, 2015 solver-independent disturbance mantığıyla uyumludur. ⁵⁵

Yüksek-throughput RL için aynı interface altında fixed-step RK4 daha uygun olabilir:

```

def rk4_step(rhs, t, x, h):
    k1 = rhs(t, x)

```

```

k2 = rhs(t + 0.5*h, x + 0.5*h*k1)
k3 = rhs(t + 0.5*h, x + 0.5*h*k2)
k4 = rhs(t + h, x + h*k3)
return x + (h / 6.0) * (k1 + 2*k2 + 2*k3 + k4)

```

Buradaki temel kural yine aynıdır: `rhs` dört kez çağrılırken dört farklı random sample çekmemelidir. Disturbance realization, step başlamadan önce fiziksel zaman tarafından belirlenmiş olmalıdır. Aksi takdirde RK4 ile Euler yalnızca farklı sayısal integrator değil, farklı stochastic plant'ler hâline gelir; 2015 revizyonunun ortaya çıkardığı sorun tam olarak budur. ³¹

Python araç zinciri açısından seçim şöyle yapılmalıdır:

Araç	Bu proje için rol	Değerlendirme
SciPy <code>solve_ivp</code>	İlk high-fidelity implementation	En düşük kurulum maliyeti; BDF/Radau/LSODA/ events/Jacobian hazır. ³⁴
SUNDIALS CVCODE	Production research simulator	ODE için en güçlü varsayılanım; stiff BDF, non-stiff Adams, Newton/Krylov/rootfinding. ³
scikits.odes	Python → CVCODE/IDA wrapper	CVCODE ve IDA wrapper'ları sağlar; native dependency/build bakımına dikkat gerekir. ⁵⁶
scikit-SUNDAE	Daha modern CVCODE/ IDA Python binding alternatifi	Güncel belgeleri doğrudan CVCODE ve IDA bindings sağladığını belirtir. ⁵⁷
Assimulo	Solver cross-check / CVCODE, IDA, Radau5, Rodas, Dopri5	Birden fazla ODE/DAE ailesini tek Python çatı altında karşılaştırmak için yararlı. ⁵⁸
CasADi	Symbolic rewrite, AD, sensitivities, NMPC	<code>cvodes</code> , <code>idas</code> , collocation ve RK integrator plugin'leri; Jacobian/sensitivity açısından çok güçlü. ⁵⁹
cypopt / IPOPT	Trajectory optimization / direct collocation	Zaman integratörü değildir; TE ODE/DAE'si discretize edilerek oluşturulan NLP'nin çözümünde kullanılmalıdır. Jockenhövel tipi dynamic optimization katmanına aittir. ²⁴
JAX + Difffrax	Differentiable, batched simulator	Non-stiff explicit ve stiff implicit yöntemler; GPU/TPU/ JIT potansiyeli. ³⁹
NumPyro	Bayesian parameter inference / uncertainty	NumPyro JAX tabanlı probabilistic programming altyapısıdır ve ODE örnekleri vardır; TE'nin ana numerical integrator'ı yerine JAX simulator etrafındaki inference katmanı olarak düşünülmelidir. ⁶⁰

CasADi ile modelin tam symbolic yeniden yazımı, bir sonraki aşamada optimal control ve differentiable learning hedefleniyorsa özellikle çekicidir. `cvodes` ve `idas` SUNDIALS plugin'lerinin yanında collocation/RK integrator'ları da vardır; CasADi expression graph'tan türev bilgisi üretebilir. ⁵⁹ Bunun bedeli, legacy Fortran'ı birebir wrapping yerine 50-state modelin tüm thermodynamic/kinetic kodunun sembolik olarak yeniden yazılması ve uzun bir parity-validation sürecidir.

JAX tarafında eski TE Fortran'ını wrap etmek yerine saf JAX model yazmak gerekir. Diffrax, stiff problemlerde implicit integrator kullanımını özellikle destekler; high-throughput batched rollouts için `jit` / `vmap` avantajları olabilir. ³⁹ Ancak 50-state tek trajectory'de implicit Newton solve + JIT compilation overhead'i CPU Fortran'dan daha hızlı olmak zorunda değildir; JAX'ın avantajı yüzlerce/binlerce paralel rollout ve gradient gerektiğinde belirginleşir. Bu nedenle önce CPU reference implementation, sonra JAX port daha düşük risklidir.

Gerçek-zaman/hızlandırma seçeneklerinde üç seviye öneriyorum. Birinci seviye hiçbir fizik kaybetmeden Fortran/C compiled RHS + fixed RK4 veya CVODE kullanmaktır; jkitchin reposu kendi Fortran backend'i için saf Python'a göre yaklaşık 5–10× hız avantajı bildirir. ⁴⁹ İkinci seviye, action interval başına bir learned discrete transition

$$\hat{x}_{k+1} = g_{\theta}(x_k, u_k, d_k)$$

öğrenmektir. Bu çok hızlıdır fakat fault/shutdown bölgelerinde extrapolation riski vardır. Üçüncü seviye ise local linear/model-order-reduced modellerdir; TE'nin altı operating mode'a sahip nonlinear yapısı nedeniyle tek global linear surrogate yerine mode-conditioned veya local models daha güvenlidir. Altı proses modu 1993 probleminin parçasıdır. ⁶¹

En güvenli RL yaklaşımı **multi-fidelity**'dir: milyonlarca training step'i validated fixed RK4 veya surrogate üzerinde üretmek; periyodik policy evaluation ve final benchmark'ı CVODE-BDF/Radau reference simulator'da yapmak. Domain randomization'a discretization model error da eklenebilir. Böylece policy yalnız tek integrator'ın sayısal artefact'larını öğrenmez.

Doğrulama, benchmark ve tekrarlanabilirlik

TE gibi benchmark bir proseste “solver başarılı döndü” fidelity kanıtı değildir. En az dört ayrı şeyi doğrulamak gerekir: fizik eşdeğerliği, numerical convergence, stochastic reproducibility ve Gymnasium semantics.

İlk referans **orijinal Fortran** olmalıdır. Downs–Vogel `TEINIT` başlangıç durumu ve türevlerini sağlar; ayrıca `TIME=0` durumunda measurement-noise davranışının özel kullanımı ve steady-state hesabı için derivative residual'ları kullanma yaklaşımı açıklanmıştır. ⁶¹ Yeni Python RHS, aynı state ve MV girişlerinde Fortran `TEFUNC` ile component-wise derivative ve measurement parity testine tabi tutulmalıdır. Bu test integrator'dan bağımsızdır ve denklem portundaki hataları en hızlı yakalayan testtir.

Önerilen test matrisi:

Test	Referans	Ölçüt	Önerilen kabul kriteri
RHS parity	Original Fortran	$\ f_{py} - f_F\ $	Double precision ölçeğinde, mümkünse 10^{-10} – 10^{-8} relative
Initial steady state	<code>TEINIT</code> / nominal operation	$\ \dot{x}\ _{\text{scaled}}$	State ölçeğine göre near-zero; Fortran ile eşleşme
1993 open-loop perturbation	Paper'daki cooling-water step experiment	reactor pressure/ temp trajectory	Plot ve NRMSE karşılaştırması; paper'ın Euler 1 s koşulu ayrıca tekrarlanmalı. ⁶²

Test	Referans	Ölçüt	Önerilen kabul kriteri
Step-halving convergence	Aynı solver	$h, h/2, h/4$ trajectory	Beklenen mertebede hata azalması
Cross-solver	Tight Radau/ CVODE-BDF oracle	NRMSE/max error	Training solver için kritik measured variables'da tercihen 10^{-3} veya daha iyi normalized error
Safety event	Tight implicit oracle	shutdown time	Fark internal step'ten küçük
Stochastic invariance	Aynı pre-generated disturbance path	solverler arası disturbance sequence	Bitwise aynı input realization
Ensemble statistics	Revised TE	mean/covariance/quantiles	Monte Carlo confidence intervals içinde
Gym API	Gymnasium checker	<code>check_env</code> , spaces, seeding	Sıfır API hatası. ⁶³
Performance	Aynı CPU/GPU	transitions/s, RHS evals	Accuracy koşulunu sağlayan en hızlı backend

Tablodaki sayısal kabul eşikleri literatürün zorunlu TE standartları değil, bu proje için önerilen engineering gates'tir. Özellikle legacy Euler 1 s ile 10^{-3} trajectory hedefinin her fault senaryosunda karşılanacağı varsayılmamalıdır; ölçülmelidir.

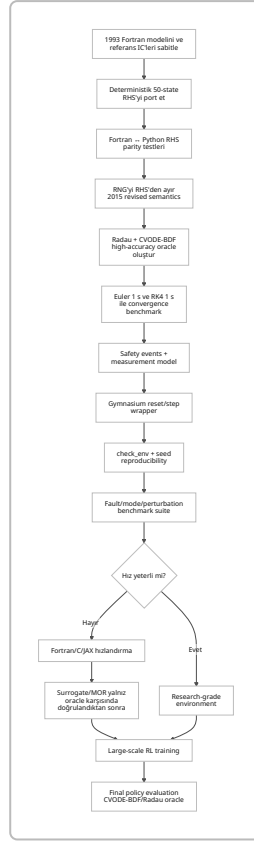
2015 revizyonu ayrıca mükemmel bir **solver-invariance regression test** sağlar. Orijinal stochastic kodda Euler/ode3/ode45 reactor temperature ve pressure sonuçları dramatik biçimde ayrılırken revised modelde curves çakışmaktadır. Aynı test Python ortamında da yapılmalıdır: aynı disturbance seed/path altında Euler, RK4, Radau ve BDF'nin yalnız numerical truncation error kadar ayrışması gerekir; random input sequence kesinlikle solver'a göre değişmemelidir. ⁶⁴

Bathelt ve arkadaşlarının timing sonuçları performans benchmarking için de faydalı bir tarihsel referans verir. 15 h simülasyonda revised execution structure, aynı dönemin MATLAB/Simulink testlerinde original Ricker koduna kıyasla en az yaklaşık %40 daha kısa süre elde etmiş; makale toplam kazancı yaklaşık %45 olarak özetlemiştir. Ancak bunlar 2010-era MATLAB/CPU ölçümleridir ve modern Python backend performansının sayısal beklentisi olarak kullanılmamalıdır. ⁴³

Reproducibility için her episode log'unda en az şu metadata saklanmalıdır: model commit/version, solver ve versiyonu, solver toleransları, `max_step`, internal fixed step varsa değeri, action/control interval, initial-condition identifier, process-disturbance seed, measurement-noise seed, action scaling, observation scaling ve floating-point dtype. Aynı model koduyla farklı `rtol` veya control interval'ı kullanan iki çalışma aynı benchmark konfigürasyonu değildir; 2015 paper'ın solver/settings bağımlılığı bunu doğrudan göstermiştir. ²⁷

Gymnasium tarafında fiziksel plant shutdown'ı ile yapay episode horizon'ı ayrılmalıdır: safety/physical envelope ihlali `terminated`, training time limit'i ise `truncated` semantiğine karşılık gelir. Gym/Gymnasium geçişinde reset API'si de `(obs, info)` biçimine dönmüştür; özel ortam testleri `check_env` ile CI içinde çalıştırılmalıdır. ⁶⁵

Uygulama akışı aşağıdaki gibi olmalıdır:



Bu sıranın avantajı, “önce Gym wrapper, sonra fizik hatalarını arama” yerine model doğruluğunu RL altyapısından bağımsız biçimde ispatlamasıdır.

Nihai öneri ve karar matrisi

Benim önerdiğim canonical mimari, 2015'te düzeltilmiş stochastic semantics ile **1993 Downs–Vogel 50-state ODE fizik çekirdeğinin** birleşimidir. Başka bir deyişle: kimya/thermodynamics/flows için original Fortran equations; random disturbances/noise ve execution scheduling için Bathelt–Ricker yaklaşımı; RL interface'i için modern Gymnasium; numerical backend için değiştirilebilir solver abstraction. Bu, hem literatürle karşılaştırılabilirlik hem modern solver doğruluğu sağlar. ⁶⁶

Birincil production/reference solver olarak CVODE-BDF'yi seçerdim. Nedeni TE'nin “mutlaka sert DAE” olması değil; ODE olmasına rağmen çok zaman ölçekli nonlinear plant olması, fault/safety senaryolarında robustness gerektirmesi ve CVODE'nun stiff BDF 1–5, Newton, Jacobian, sparse/Krylov ve rootfinding desteğini olgun tek bir çözümde birleştirmesidir. SUNDIALS stiff problemler için açıkça BDF, non-stiff için Adams seçimini önerir. ³⁷ 50 state için başlangıçta dense Newton kullanır, ancak profiling büyük controller/observer augmentation'da fayda gösterirse sparse/Krylov'a geçerdim.

İkinci, bağımsız oracle solver SciPy Radau olmalıdır. İki farklı implicit method ailesinin aynı trajectory'ye yakınsaması, tek solver'a güvenmekten daha güçlü validation sağlar.

`Radau(rtol=1e-9, scaled_atol≈1e-11, max_steps≤1s)` gibi sıkı bir konfigürasyon, production solver değil reference trajectory üreticisi olarak uygundur. SciPy Radau'yu stiff problemler için önerir ve event/Jacobian desteği sağlar. ⁴

Yüksek hacimli RL training default'u olarak fixed-step RK4, 1 s öneriyorum. Bunun gerekçesi Euler'in yalnızca birinci merteye olması, fakat TE'nin tarihsel 1 s time scale'inin çok değerli bir güvenlik anchor'ı sağlamasıdır. RK4 dört RHS değerlendirmesine mal olur, ancak adaptive solver'in episode-to-episode farklı iteration sayısı, Newton solves ve allocator overhead'inden kaçınır. Önce oracle karşısında step-halving yapılmalı; RK4 2 s veya 5 s aynı fidelity eşiğini geçerse adım sonradan artırılmalıdır.

Başlangıçtan doğrudan 5–10 s kullanmak tavsiye edilmez. Euler 1 s ise legacy reproducibility ve çok ucuz baseline olarak korunmalıdır. 1993 ve 2015 sonuçları 1 s Euler'in özel tarihsel rolünü destekler. ⁶⁷

Adaptive explicit RK45/DOP853, stochastic scheduler tamamen RHS'den ayrıldıktan ve stiffness profiling yapıldıktan sonra iyi bir ara seçenek olabilir. DOP853 özellikle high-accuracy non-stiff çözüm için SciPy'nin önerdiği explicit yöntemdir; fakat fault regimes'de implicit BDF/Radau kadar güvenli varsayılmalıdır. ³⁴

IDA'yı varsayılan TE çözücüsü yapmazdım. Original 1993 model DAE değildir ve IDA kullanmak ek residual/Jacobian/consistent-initial-condition karmaşıklığı yaratır. IDA ancak Modelica/first-principles yeniden formülasyonu cebirsel variables'ı explicit olarak koruduğunda veya Jockenhövel tipi DAE modeline geçildiğinde doğru araçtır. IDA'nın resmi formulation'ı $F(t, y, \dot{y}) = 0$ ve variable-order BDF 1–5'tir; semi-explicit index-one sınıfı için consistent initial conditions mekanizması vardır. ⁶⁸

CasADi, amacın yalnız RL değil aynı model üzerinde differentiable policy optimization, system identification veya NMPC olması durumunda en stratejik ikinci implementasyondur. CVODES/IDAS ve automatic differentiation kombinasyonu burada güçlüdür. ³⁸ `cyipopt` ise simulator backend olarak değil, collocation/multiple-shooting sonrası oluşan nonlinear programın optimizier'ı olarak kullanılmalıdır. Jockenhövel tipi TE dynamic optimization çalışmaları bu katman ayrımının doğal örneğidir. ²⁴

JAX/Diffrax, binlerce parallel differentiable rollout gerektiğinde üçüncü backend olarak değerlidir. Stifflik doğrulanırsa Kvaerno ailesi gibi implicit yöntemler, non-stiff kanıtlanırsa explicit solver tercih edilebilir; Diffrax'ın kendi kılavuzu da stiff problemlerde daha maliyetli implicit yöntemlere geçilmesi gerektiğini belirtir. ⁶⁹ NumPyro bunun üstünde parameter uncertainty, Bayesian calibration veya posterior predictive simulation için kullanılmalıdır; ana plant integrator'ın yerine geçmez. ⁶⁰

Son karar matrisi:

Senaryo	Plant modeli	Varsayılan solver	Fallback	İç adım / tolerans	Hüküm
Yayın kalitesinde RL evaluation	Full 1993 + revised RNG	CVODE-BDF	Radau	<code>rtol 1e-7, scaled atol 1e-9, başlangıç max_step 1s</code>	En güçlü genel seçim
Python-only prototip	Full 1993	SciPy BDF	Radau/LSODA	aynı	En düşük geliştirme maliyeti
Ground-truth trajectory	Full 1993 deterministic	Radau tight	CVODE-BDF tight	<code>rtol≈1e-9, scaled atol≈1e-11, ≤1 s</code>	Oracle

Senaryo	Plant modeli	Varsayılan solver	Fallback	İç adım / tolerans	Hüküm
On-policy milyonlarca step	Full 1993	RK4 fixed 1 s	Euler 1 s	convergence sonrası artırılabilir	Önerilen training default
Legacy Downs-Vogel parity	Original semantics	Euler fixed 1 s	—	1 s	Tarihsel referans
Gerçek zaman / embedded-benzeri rollout	Full veya reduced	RK4 / validated surrogate	Euler	CPU bütçesine göre	Hız-fidelity trade-off
Modelica / explicit DAE formulation	DAE	IDA/IDAS	DASSL	consistent IC + vector tolerances	DAE için doğru seçim
Differentiable RL/NMPC	Symbolic TE	CasADi CVODES/ IDAS	DiffraX	sensitivity-aware	Gradient gereken durum
Massively batched JAX training	JAX TE	DiffraX	fixed JAX RK4	validation'a göre	GPU/ batching senaryosu
Surrogate training	learned transition	Neural/ local reduced model	RK4 full model	oracle ile sürekli audit	Yalnız multi-fidelity kullanım

En kritik üç engineering kuralı şunlardır: **orijinal TE'yi gereksiz yere DAE'ye çevirmeyin; RNG'yi integrator RHS çağrılarında tamamen ayırın; training solver'ını tight CVODE/Radau oracle'a karşı convergence-test etmeden hızlandırmayın.** Bunlar uygulandığında Gymnasium katmanı sayısal modele zarar veren bir abstraction olmaktan çıkar ve yalnız sampled-data RL interface'i hâline gelir. Orijinal TE'nin 50-state ODE interface'i, 2015 solver-independence düzeltmesi ve modern SUNDIALS/ SciPy capabilities bu mimariyi birlikte desteklemektedir. ⁷⁰

¹ ⁵ ¹⁰ ¹¹ ¹² ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²⁰ ²⁶ ³³ ⁶¹ ⁶² ⁶⁶ ⁶⁷ ⁷⁰ PII: 0098-1354(93)80018-I

<https://users.abo.fi/khaggblo/RS/Downs.pdf>

² ⁸ ²¹ ²⁷ ²⁹ ³⁰ ³¹ ⁴³ ⁴⁸ ⁵⁵ ⁶⁴ <https://skoge.folk.ntnu.no/prost/proceedings/adchem2015/media/papers/0010.pdf>

<https://skoge.folk.ntnu.no/prost/proceedings/adchem2015/media/papers/0010.pdf>

³ ³² ³⁷ https://sundials.readthedocs.io/en/latest/cvode/Mathematics_link.html

https://sundials.readthedocs.io/en/latest/cvode/Mathematics_link.html

⁴ ³⁴ ³⁵ ³⁶ ⁴⁰ ⁵⁴ https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html

https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html

⁶ ⁶⁸ https://sundials.readthedocs.io/en/latest/ida/Mathematics_link.html

https://sundials.readthedocs.io/en/latest/ida/Mathematics_link.html

7 38 <https://web.casadi.org/docs/>
<https://web.casadi.org/docs/>

9 44 49 <https://github.com/jkitchin/tennessee-eastman-profbraatz>
<https://github.com/jkitchin/tennessee-eastman-profbraatz>

13 14 15 https://raw.githubusercontent.com/jkitchin/tennessee-eastman-profbraatz/master/src/tep/python_backend.py
https://raw.githubusercontent.com/jkitchin/tennessee-eastman-profbraatz/master/src/tep/python_backend.py

22 <https://www.nist.gov/publications/implementations-tennessee-eastman-process-modelica>
<https://www.nist.gov/publications/implementations-tennessee-eastman-process-modelica>

23 <https://github.com/ElsevierSoftwareX/SOFTX-D-24-00664>
<https://github.com/ElsevierSoftwareX/SOFTX-D-24-00664>

24 https://www.researchgate.net/publication/222699084_Dynamic_Optimization_of_the_Tennessee_Eastman_Process_Using_the_OptControlCentre
https://www.researchgate.net/publication/222699084_Dynamic_Optimization_of_the_Tennessee_Eastman_Process_Using_the_OptControlCentre

25 53 63 <https://gymnasium.farama.org/api/utils/>
<https://gymnasium.farama.org/api/utils/>

28 <https://openmodelica.org/doc/OpenModelicaUsersGuide/latest/solving.html>
<https://openmodelica.org/doc/OpenModelicaUsersGuide/latest/solving.html>

39 https://docs.kidger.site/diffrax/examples/stiff_ode/
https://docs.kidger.site/diffrax/examples/stiff_ode/

41 42 <https://sundials.readthedocs.io/en/latest/cvode/Usage/index.html>
<https://sundials.readthedocs.io/en/latest/cvode/Usage/index.html>

45 <https://github.com/camaramm/tep2py>
<https://github.com/camaramm/tep2py>

46 51 https://pytep.readthedocs.io/en/latest/readme_link.html
https://pytep.readthedocs.io/en/latest/readme_link.html

47 https://github.com/AIRI-Institute/iprl_benchmark/blob/main/README.md
https://github.com/AIRI-Institute/iprl_benchmark/blob/main/README.md

50 <https://raw.githubusercontent.com/jkitchin/tennessee-eastman-profbraatz/master/src/tep/simulator.py>
<https://raw.githubusercontent.com/jkitchin/tennessee-eastman-profbraatz/master/src/tep/simulator.py>

52 <https://github.com/kennyuren/COSTEP>
<https://github.com/kennyuren/COSTEP>

56 <https://bmcage.github.io/odes/master/api/compat.html>
<https://bmcage.github.io/odes/master/api/compat.html>

57 <https://scikit-sundae.readthedocs.io/en/stable/api/sksundae/>
<https://scikit-sundae.readthedocs.io/en/stable/api/sksundae/>

58 <https://github.com/modelon-community/Assimulo>
<https://github.com/modelon-community/Assimulo>

59 https://web.casadi.org/api/internal/dd/d1b/group__integrator.html
https://web.casadi.org/api/internal/dd/d1b/group__integrator.html

⁶⁰ <https://github.com/pyro-ppl/numpyro>

<https://github.com/pyro-ppl/numpyro>

⁶⁵ https://gymnasium.farama.org/gym_release_notes/index.html

https://gymnasium.farama.org/gym_release_notes/index.html

⁶⁹ <https://docs.kidger.site/diffrax/usage/how-to-choose-a-solver/>

<https://docs.kidger.site/diffrax/usage/how-to-choose-a-solver/>